Lab 06: Parsing Static Web Pages

INFO 4617/5617 · Week 6 · Chapter 6, Parsing Static Web Pages

This week every class is live coding: one lab in three parts, one a day. Each step says what we build and why. Its code cells start with a comment that says what to write, and we write it together. Cells that start `# Given:` are already written: we read them together and run them. Nothing here is turned in.

We turn web pages into tables, on sites that allow it: Wikipedia's List of highest-grossing films, the article on *The Godfather* in three languages, and Quotes to Scrape, a site built for practice:

- **Part 1, Monday:** ask first; fetch one page and save it; read every table on it in one call; clean a column, and check what the cleaning did;
- **Part 2, Wednesday:** find a table by its tag and class and read it by hand; then a film's infobox, in English, French and Spanish;
- **Part 3, Friday:** turn boxes that repeat on a page into the rows of a table, page after page; then a page whose data isn't in its HTML.

Short ⏱️ microsprints ask you to change one thing, alone or in pairs, and share what you find. After each step, a 💬 **Discuss** box holds questions to talk over before the next one; click it to open it.

Before class: open this notebook in the `webdata` environment (week 1), and have your email address ready for the User-Agent. Wednesday and Friday use Chrome's Inspector (week 5).

On Wednesday and Friday, start at that day's setup step.

> **Instructor key.** Rebuilt from scratch on 2026-11-11 for the next offering, from chapter 6 in the textbook at commit `3036573`, after the audit in issue 180. Run with `labs/tools/run_key.py` on 2026-11-11 from 06:51 to 06:52 UTC, in Python 3.14.6 with pip's pandas 3.1.1, beautifulsoup4 4.15.0 and requests 2.34.2. Every cell ran, and none is tagged `not-run-live`: Wikimedia and Quotes to Scrape answered every request. Step 4's microsprint raises an `ImportError` on purpose (its cell is tagged `raises-exception`); where `html5lib` is installed, the error is `ValueError: No tables found` instead. The saved copies in `labs/week-06/data/` were written by this run's own cells, with `saved` pointed at that folder (served on the machine as raw.githubusercontent.com serves it, as UTF-8 text) and the file names at its path for the run, then set back to `main`. The lab requests nothing from The Numbers, oscars.org, Rotten Tomatoes, Box Office Mojo or IMDb but their `robots.txt`. A rerun should match the Godfather infoboxes and Quotes to Scrape unless they're edited. It won't necessarily match the list of highest-grossing films, which changes while films are in theaters (a 2026 film was third), nor Step 1's answers: The Numbers refused this machine's `robots.txt` request with 403 in this run, and answered 200 earlier the same morning.

Part 1 — Monday: one table, from a web page to checked numbers

We ask before we fetch, fetch one page once and keep it, and let pandas read every table on it. Then we clean one column of money, and check the result, because a cleanup can be wrong without any error.

Step 0 — Set up (3 minutes)

Put your email address in `CLIENT`, then run the cell. It imports what we need and defines `HEADERS`, the User-Agent every request sends: the course's name, your address, and the library's name and version, which Wikimedia's User-Agent policy asks for. `saved` is the address of the course's saved copies, the **checkpoints**: if a site refuses you, or you missed a day, you go on from those.

```
# Given: Part 1's setup. Put your email address in CLIENT, then run it
import time
from io import StringIO
from urllib.robotparser import RobotFileParser

import requests
import pandas as pd
import matplotlib.pyplot as plt

CLIENT = "WebDataScience/1.0 (your-email@colorado.edu)"
HEADERS = {"User-Agent": f"{CLIENT} python-requests/{requests.__version__}"}
LIST_URL = "https://en.wikipedia.org/wiki/List_of_highest-grossing_films"
saved =
 ↪  "https://raw.githubusercontent.com/cuinfoscience/INFO4617-Fall2026/main/labs/week-06/data/"
```

💬 Discuss

- Every request today carries your email address. Who at a site might read it, and what could they do with it?
- Week 2 called naming yourself a courtesy. What does it let a site do that it couldn't do with an anonymous request?

Step 1 — Check before you scrape (9 minutes)

A site's `robots.txt` says which pages crawlers, programs that fetch pages automatically, may fetch. Week 2 read it with `RobotFileParser`. Ask for Wikipedia's, and print its status first: a parser handed an error page finds no rules in it, and then says yes to everything. Then parse it and ask about our page. The given loop asks the chapter's six other sites the same question, a second apart.

```
# wiki_robots: request "https://en.wikipedia.org/robots.txt" with headers=HEADERS
# Print its status_code and the length of its text
wiki_robots = requests.get("https://en.wikipedia.org/robots.txt", headers=HEADERS)
print(wiki_robots.status_code, len(wiki_robots.text))
```

```
200 28028
```

```
# wiki_rules: a RobotFileParser() that parses wiki_robots.text.splitlines()
# Print whether CLIENT may fetch LIST_URL: wiki_rules.can_fetch(CLIENT, LIST_URL)
wiki_rules = RobotFileParser()
wiki_rules.parse(wiki_robots.text.splitlines())
print(wiki_rules.can_fetch(CLIENT, LIST_URL))
```

```
True
```

```
# Given: ask the chapter's six other sites about its page there, a second apart. The host is
# the third piece of the address split on "/". Trust "may fetch" only on a line starting 200
CHAPTER_PAGES = ["https://www.boxofficemojo.com/weekend/2018W52/",
                 "https://www.the-numbers.com/box-office-chart/weekly/2018/12/28",
                 "https://www.oscars.org/oscars/ceremonies/2026",
                 "https://www.rottentomatoes.com/browse/movies_in_theaters/",
                 "https://www.imdb.com/title/tt0068646/",
                 "https://quotes.toscrape.com/"]
for page in CHAPTER_PAGES:
    host = page.split("/")[2]
    robots = requests.get(f"https://{host}/robots.txt", headers=HEADERS)
    rules = RobotFileParser()
    rules.parse(robots.text.splitlines())
```

```
    print(robots.status_code, host, "may fetch:", rules.can_fetch(CLIENT, page))
    time.sleep(1)
```

```
200 www.boxofficemojo.com may fetch: False
403 www.the-numbers.com may fetch: True
200 www.oscars.org may fetch: True
200 www.rottentomatoes.com may fetch: True
200 www.imdb.com may fetch: False
404 quotes.toscrape.com may fetch: True
```

💬 **Discuss**

- Two sites said `False`. For the others, does `True` mean the site wants us to scrape it? (Step 2.)
- In the key's run, The Numbers refused the `robots.txt` request itself (403), and the parser said `True`. Python's own `RobotFileParser.read()` reads a 401 or 403 as "fetch nothing": which reading would you follow?
- Typed without `.splitlines()`, `parse()` reads the text a character at a time and finds no rules; with `can_fetch()`'s two arguments swapped, it finds none that match. Both say `True` to IMDb, with no error. How would you notice?

Step 2 — Read one site's terms (5 minutes)

`robots.txt` speaks to crawlers. A site's **terms of use**, the contract it offers its visitors, can say more. Open The Numbers' Terms of Service in your browser and read section 4, *Automated Access, Scraping, and Bots*. The terms of oscars.org and Rotten Tomatoes say much the same. So this lab sends those three sites only Step 1's `robots.txt` request, and scrapes Wikipedia and Quotes to Scrape, which was made for practice.

💬 **Discuss**

- The Numbers' `robots.txt` said yes to its chart page, and its terms say scraping is "prohibited". Week 2 called terms of service "quasi-law": which of the two binds us, and which is a request?
- Section 4.4 asks every bot to put the word "bot" in its User-Agent. Is one request sent from a notebook a bot?
- Thirty requests, one from each of us, are no load on a server. Is load the only reason a site forbids scraping?

Step 3 — Fetch the page once, look, and save it (8 minutes)

Request the list once, and look at what came back before reading it: the status, the length, and the first 200 characters. A refusal (403 or 429) shows up here, and the given cell under it loads the course's copy. Then save the text to a file and read it back: from now on we parse the file, so Wikipedia is asked once. `with open(...) as f:` opens a file, and closes it after the indented lines.

```
# list_page: request LIST_URL with headers=HEADERS
list_page = requests.get(LIST_URL, headers=HEADERS)
```

```
# Look first: print the status and len(list_page.text); then its first 200 characters
print(list_page.status_code, len(list_page.text))
print(list_page.text[:200])
```

```
200 2213823
<!DOCTYPE html>
<html class="client-nojs vector-feature-language-in-header-enabled
 ↪  vector-feature-language-in-main-menu-disabled
 ↪  vector-feature-language-in-main-page-header-disabled vector-feature-pag
```

```
# Given, only if the cell above didn't print 200: remove the "#" below (saved 2026-11-11)
# list_page = requests.get(saved + "list_page.html", headers=HEADERS)
```

```
# Save the page: with open("list_page.html", "w", encoding="utf-8") as f:
#   then, indented under it, f.write(list_page.text)
with open("list_page.html", "w", encoding="utf-8") as f:
    f.write(list_page.text)
```

```
# Read it back: with open("list_page.html", encoding="utf-8") as f:
#   then, indented, list_html = f.read(); after that, unindented, print len(list_html)
with open("list_page.html", encoding="utf-8") as f:
    list_html = f.read()
print(len(list_html))
```

```
2213823
```

💬 Discuss

- The file holds the article as it was today. What can't a saved copy tell you that a request next month could?
- Sent without `headers=HEADERS`, the request got a 403 and 126 characters: "Please set a user-agent and respect our robot policy". What would the file hold then, and which cell would fail first?
- Wikimedia's Robot policy asks bots to use `/wiki/Title` addresses with no `?` after them, because those can come from a cache. Why is a copy from a cache cheaper for Wikipedia than a fresh page?

Step 4 — Every table in one call (10 minutes)

`pd.read_html()` finds every `<table>` in a page and returns a list of DataFrames, one each. It reads a file, not a string, so `StringIO()` makes our text look like a file. A Wikipedia list has many tables: count them, then take the first and look (chapter, Strategy 3). If you ever see `ImportError: `Import html5lib` failed`, don't install anything: the text had no table in it.

```
# list_tables: pd.read_html(StringIO(list_html)); print how many tables it found
list_tables = pd.read_html(StringIO(list_html))
print(len(list_tables))
```

```
93
```

```
# films: the first table, list_tables[0]; show films.head()
films = list_tables[0]
films.head()
```

	Rank	Peak	Title	Worldwide gross	Year	Ref
0	1	1	Avengers: Endgame †	$2,942,649,778	2019	[# 1][# 2]
1	2	1	Avatar	$2,923,710,708	2009	[# 3][# 4]
2	3	3	Spider-Man: Brand New Day †	$2,523,147,515	2026	[# 5][# 6]
3	4	3	Avatar: The Way of Water	$2,334,484,620	2022	[# 7][# 8]
4	5	1	Titanic	T$2,257,906,828	1997	[# 9][# 10]

⏱️ Microsprint (4 minutes, alone or in pairs)

Do: hand `read_html()` a text with no table in it: Step 1's `wiki_robots.text`. Read the last line of the error. **Share:** what the error tells you to do, and what actually went wrong.

```
# Microsprint: pd.read_html(StringIO(wiki_robots.text)); read the error's last line
pd.read_html(StringIO(wiki_robots.text))
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
File /opt/webdata/lib/python3.14/site-packages/pandas/compat/_optional.py:158, in
 ↪  import_optional_dependency(name, extra, min_version, errors)
    157 try:
--> 158     module = importlib.import_module(name)
    159 except ImportError as err:

File ~/.local/share/uv/python/cpython-3.14.6-linux-x86_64-gnu/lib/python3.14/importlib/__init__.
 ↪  py:88, in import_module(name,
 ↪  package)
     87         level += 1
---> 88 return _bootstrap._gcd_import(name[level:], package, level)

File <frozen importlib._bootstrap>:1406, in _gcd_import(name, package, level)
   1404     if level > 0:
   1405         name = _resolve_name(name, package, level)
-> 1406     return _find_and_load(name, _gcd_import)

File <frozen importlib._bootstrap>:1371, in _find_and_load(name, import_)
   1369             module = sys.modules.get(name, _NEEDS_LOADING)
   1370             if module is _NEEDS_LOADING:
-> 1371                 return _find_and_load_unlocked(name, import_)
   1372

File <frozen importlib._bootstrap>:1335, in _find_and_load_unlocked(name, import_)
   1333     spec = _find_spec(name, path)
   1334     if spec is None:
-> 1335         raise ModuleNotFoundError(f'{_ERR_MSG_PREFIX}{name!r}', name=name)
   1336     else:

ModuleNotFoundError: No module named 'html5lib'

The above exception was the direct cause of the following exception:

ImportError                               Traceback (most recent call last)
Cell In[12], line 2
      1 # Microsprint: pd.read_html(StringIO(wiki_robots.text)); read the error's last line
----> 2 pd.read_html(StringIO(wiki_robots.text))

File /opt/webdata/lib/python3.14/site-packages/pandas/io/html.py:1234, in read_html(io, match,
 ↪  flavor, header, index_col, skiprows, attrs, parse_dates, thousands, encoding, decimal,
 ↪  converters, na_values, keep_default_na, displayed_only, extract_links, dtype_backend,
 ↪  storage_options)
   1230 check_dtype_backend(dtype_backend)
   1232 io = stringify_path(io)
-> 1234 return _parse(
   1235     flavor=flavor,
   1236     io=io,
   1237     match=match,
   1238     header=header,
   1239     index_col=index_col,
   1240     skiprows=skiprows,
   1241     parse_dates=parse_dates,
   1242     thousands=thousands,
   1243     attrs=attrs,
```

```
   1244     encoding=encoding,
   1245     decimal=decimal,
   1246     converters=converters,
   1247     na_values=na_values,
   1248     keep_default_na=keep_default_na,
   1249     displayed_only=displayed_only,
   1250     extract_links=extract_links,
   1251     dtype_backend=dtype_backend,
   1252     storage_options=storage_options,
   1253 )

File /opt/webdata/lib/python3.14/site-packages/pandas/io/html.py:966, in _parse(flavor, io,
 ↪  match, attrs, encoding, displayed_only, extract_links, storage_options, **kwargs)
    964 retained = None
    965 for flav in flavor:
--> 966     parser = _parser_dispatch(flav)
    967     p = parser(
    968         io,
    969         compiled_match,
   (...)    974         storage_options,
    975     )
    977     try:

File /opt/webdata/lib/python3.14/site-packages/pandas/io/html.py:910, in _parser_dispatch(flavor)
    905     raise ValueError(
    906         f"{flavor!r} is not a valid flavor, valid flavors are {valid_parsers}"
    907     )
    909 if flavor in ("bs4", "html5lib"):
--> 910     import_optional_dependency("html5lib")
    911     import_optional_dependency("bs4")
    912 else:

File /opt/webdata/lib/python3.14/site-packages/pandas/compat/_optional.py:161, in
 ↪  import_optional_dependency(name, extra, min_version, errors)
    159 except ImportError as err:
    160     if errors == "raise":
--> 161         raise ImportError(msg) from err
    162     return None
    164 # Handle submodules: if we have submodule, grab parent module from sys.modules

ImportError: `Import html5lib` failed.  Use pip, conda, or your preferred package management tool
 ↪  to install the html5lib package.
```

💬 Discuss

- We took table `[0]` because `head()` showed the right columns. If the article gained a table above it, what would `[0]` give you, and would anything warn you? (`read_html`'s `match=` chooses by text instead.)
- The error told you to install a package. Why is that the wrong fix here, and what would you print first?
- Week 4 parsed XML whose tags named their contents. What does a `<table>` tell `read_html`, and what doesn't it?

Step 5 — Clean a column, and check it (9 minutes)

A number stored as text can't be added or plotted. `dtypes` shows each column's type: `str` is text, `int64` a whole number. The chapter cleans `Worldwide gross` with a **regular expression**, a pattern for

text: `[^0-9]` means any character that isn't a digit. The given cell removes every such character and converts the rest with `pd.to_numeric()`. Then we check: a plot, and a sort.

```
# Print films.dtypes: which columns are text?
print(films.dtypes)
```

```
Rank                 str
Peak                 str
Title                str
Worldwide gross      str
Year               int64
Ref                  str
dtype: object
```

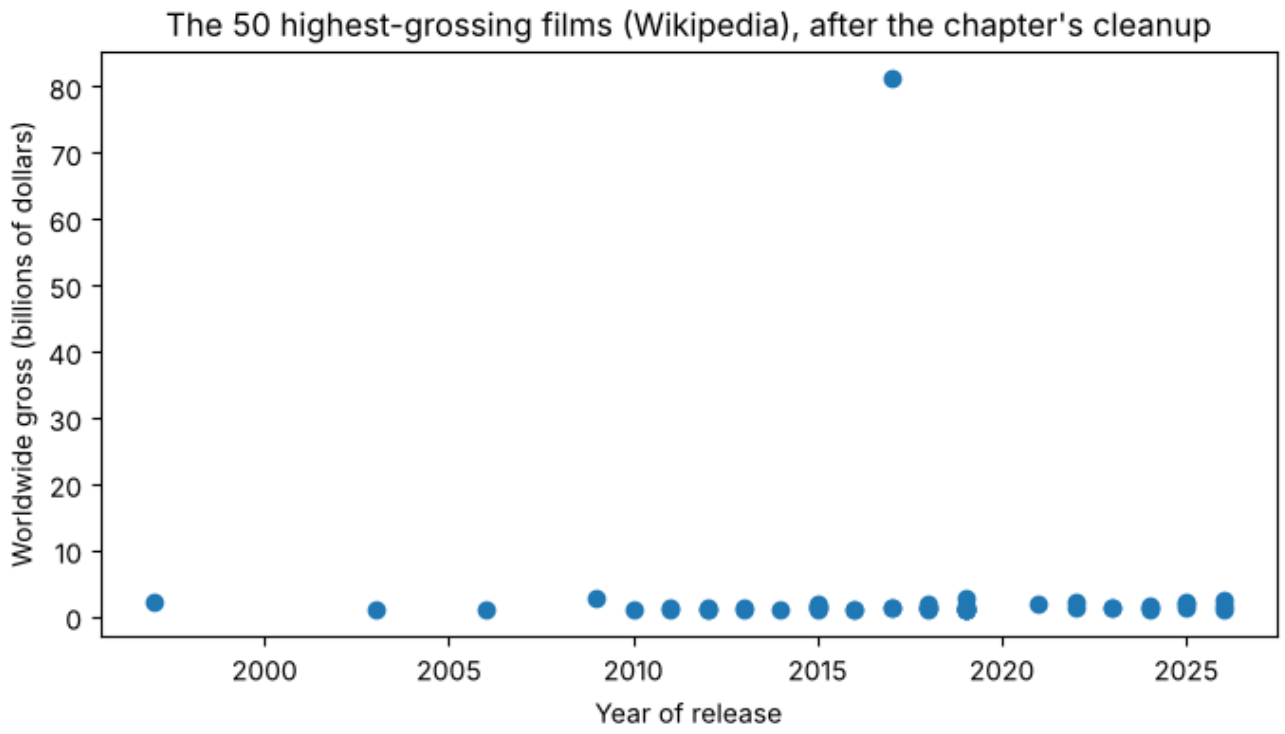
```
# Print the first ten values of films["Worldwide gross"]. Which don't start with "$"?
print(films["Worldwide gross"].head(10))
```

```
0      $2,942,649,778
1      $2,923,710,708
2      $2,523,147,515
3      $2,334,484,620
4     T$2,257,906,828
5    NZ$2,217,758,382
6      $2,068,223,624
7      $2,048,359,754
8    SM$1,922,598,800
9      $1,870,309,291
Name: Worldwide gross, dtype: str
```

```
# Given: the chapter's cleanup. Remove each character that isn't a digit (the pattern [^0-9]),
# then make the text a number. errors="coerce" makes a cell it can't read NaN, without stopping
digits = films["Worldwide gross"].str.replace(r"[^0-9]", "", regex=True)
films["gross"] = pd.to_numeric(digits, errors="coerce")
print(films[["Title", "Worldwide gross", "gross"]].head(6))
print("gross is now", films["gross"].dtype)
```

```
                        Title   Worldwide gross       gross
0          Avengers: Endgame †    $2,942,649,778  2942649778
1                       Avatar    $2,923,710,708  2923710708
2  Spider-Man: Brand New Day †    $2,523,147,515  2523147515
3     Avatar: The Way of Water    $2,334,484,620  2334484620
4                      Titanic   T$2,257,906,828  2257906828
5                    Ne Zha 2  NZ$2,217,758,382  2217758382
gross is now int64
```

```
# Given: plot each film's cleaned gross against its year of release
fig, ax = plt.subplots(figsize=(8, 4))
ax.scatter(films["Year"], films["gross"] / 1e9)
ax.set_title("The 50 highest-grossing films (Wikipedia), after the chapter's cleanup")
ax.set_xlabel("Year of release")
ax.set_ylabel("Worldwide gross (billions of dollars)")
plt.show()
```




```
# Sort films by "gross", largest first (ascending=False), and show the top three
films.sort_values("gross", ascending=False).head(3)
```

```
   Rank Peak                    Title   Worldwide gross  Year           Ref  \
30   31   11  The Fate of the Furious  F8$1,238,764,765  2017  [# 56][# 54]
0     1    1      Avengers: Endgame †    $2,942,649,778  2019    [# 1][# 2]
1     2    1                   Avatar    $2,923,710,708  2009    [# 3][# 4]

          gross
30  81238764765
0    2942649778
1    2923710708
```

💬 Discuss

- One point sits far above the rest. Which film is it, and what did the cleanup do to its `F8$1,238,764,765`?
- `errors="coerce"` turns what it can't read into NaN. Why couldn't it help here, and what kind of mistake can no setting catch?
- Would `films.head()` have shown you the problem? What does that say about how to check a cleaned column?

Step 6 — Fix the cleanup, and see what changed (6 minutes)

The stray letters are footnote markers glued to the front of the number. The given fix keeps only what follows the `$`: `str.extract()` returns the part of the text that matches the pattern's parentheses, and `\$([\d,]+)` means digits and commas after a dollar sign. Then compare the two columns: `films["gross"] != films["gross_fixed"]` is a column of `True` and `False`, and `films[changed]` keeps the rows where it's `True`.

```
# Given: keep the digits and commas after "$" (the part in parentheses); drop commas; convert
after_dollar = films["Worldwide gross"].str.extract(r"\$([\d,]+)", expand=False)
films["gross_fixed"] = pd.to_numeric(after_dollar.str.replace(",", ""), errors="coerce")
```

```
print(films[["Title", "Worldwide gross", "gross_fixed"]].head(6))
```

```
                        Title   Worldwide gross  gross_fixed
0          Avengers: Endgame †    $2,942,649,778   2942649778
1                      Avatar    $2,923,710,708   2923710708
2  Spider-Man: Brand New Day †    $2,523,147,515   2523147515
3     Avatar: The Way of Water    $2,334,484,620   2334484620
4                      Titanic   T$2,257,906,828   2257906828
5                    Ne Zha 2  NZ$2,217,758,382   2217758382
```

```
# changed: films["gross"] != films["gross_fixed"]; then show films[changed]
changed = films["gross"] != films["gross_fixed"]
films[changed]
```

```
   Rank Peak                    Title   Worldwide gross  Year           Ref  \
30   31   11  The Fate of the Furious  F8$1,238,764,765  2017  [# 56][# 54]

          gross  gross_fixed
30  81238764765   1238764765
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Do: print `list(films["Peak"].unique())`, each film's highest rank. Which values have letters in them? What would the chapter's digits-only cleanup turn each into, and would Step 6's fix help? **Share:** the worst one, and what it would become.

```
# Microsprint: print list(films["Peak"].unique())
print(list(films["Peak"].unique()))
```

```
['1', '3', '5', '4', '6', '9', '11', '8', '7', '16', '10', '14', '15', '20', '12', '35', '2',
 ↪  '24RK', '23RK', '5RK', '31', '32', '30', '4TS3', '44']
```

```
# Given: save the table in this folder as films.csv, as the course did for its saved copy
films.to_csv("films.csv", index=False)
print(len(films), "films saved")
```

```
50 films saved
```

💬 Discuss

- The fix assumes every cell has a `$`. What would it do with one that doesn't, and how would you find out?
- `4TS3` is Toy Story 3, which peaked at 4. No plot would make a 43 stand out. What check would?
- The letters come from the article's own markup. Is fixing our code enough, or should someone fix the article too?

End of Part 1

On Wednesday we look under `read_html()`: we find the same table ourselves, by its tag and class, and read it row by row; then we read a film's infobox in three languages. Part 2 starts with its own setup step, which loads today's saved page and table.

Part 2 — Wednesday: find it by its tag and class

`read_html()` did the finding for us. Today we do it by hand with BeautifulSoup, which reads HTML into a tree of tags that Python can search: a table by its tag and class, its rows and cells, and then a different kind of table, a film's box of facts.

Step 7 — Set up again (4 minutes)

Part 2 starts fresh, so a student who missed Monday can start here. Put your email address in `CLIENT`, and run both cells. The second loads Monday's page and cleaned table from the course's saved copies, so nobody asks Wikipedia for the list again. `BeautifulSoup` is new today.

```
# Given: Part 2's setup. Put your email address in CLIENT, then run it
import requests
import pandas as pd
from bs4 import BeautifulSoup

CLIENT = "WebDataScience/1.0 (your-email@colorado.edu)"
HEADERS = {"User-Agent": f"{CLIENT} python-requests/{requests.__version__}"}
GODFATHER_EN = "https://en.wikipedia.org/wiki/The_Godfather"
GODFATHER_FR = "https://fr.wikipedia.org/wiki/Le_Parrain_(film)"
GODFATHER_ES = "https://es.wikipedia.org/wiki/El_padrino_(película)"
saved =
 ↪  "https://raw.githubusercontent.com/cuinfoscience/INFO4617-Fall2026/main/labs/week-06/data/"
```

```
# Given: load Monday's page and table, saved by the course on 2026-11-11
list_html = requests.get(saved + "list_page.html", headers=HEADERS).text
films = pd.read_csv(saved + "films.csv")
print(len(list_html), "characters;", len(films), "films")
```

```
2213823 characters; 50 films
```

💬 Discuss

- If you were here on Monday, your own `list_page.html` is in your folder. Why might it differ from the course's copy?

Step 8 — Find a table by its tag and class (12 minutes)

`find_all("table")` returns every `<table>` tag; `find()` returns the first. A **class** is a label the page's author writes on a tag, `class="wikitable"`, and Wikipedia gives its data tables that one. Python already uses the word `class`, so BeautifulSoup spells it `class_`. When nothing matches, `find()` returns `None` and `find_all()` an empty list, `[]`, with no error. Spell one wrong on purpose, to see it.

```
# list_soup: BeautifulSoup(list_html, "html.parser"); print how many "table" tags it has
list_soup = BeautifulSoup(list_html, "html.parser")
print(len(list_soup.find_all("table")))
```

```
93
```

```
# wikitable: the first "table" with class_="wikitable"; print wikitable.get("class")
wikitable = list_soup.find("table", class_="wikitable")
print(wikitable.get("class"))
```

```
['wikitable', 'sortable', 'plainrowheaders', 'sticky-header', 'col4right', 'col5center',
 ↪  'col6center']
```

```
# Misspell it on purpose: misspelled, the first "table" with class_="wikitabel"; print it
misspelled = list_soup.find("table", class_="wikitabel")
print(misspelled)
```

```
None
```

```
# How many tables have the class "wikitable"?
# Print the length of list_soup.find_all("table", class_="wikitable")
print(len(list_soup.find_all("table", class_="wikitable")))
```

```
89
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Do: our table has seven classes. Count the tables that carry `"sortable"`, and those that carry `"sticky-header"`. **Share:** the two counts, and whether any class picks our table out alone.

```
# Microsprint: print how many tables have class_="sortable", then class_="sticky-header"
print(len(list_soup.find_all("table", class_="sortable")))
print(len(list_soup.find_all("table", class_="sticky-header")))
```

```
1
4
```

💬 Discuss

- `None` came back with no error. If the next line were `misspelled.find_all("tr")`, what would Python say, and where would you look first: your code, or the Inspector?
- `find()` gave us the first of many wikitables. What makes "the first" a fragile way to choose, and what did the microsprint suggest instead?

Step 9 — Rows and cells, by hand (12 minutes)

Inside a table, `<tr>` is a row, `<th>` a header cell and `<td>` an ordinary cell. Find the rows; then the first row's cells, with `find_all(["th", "td"])`: a list asks for either tag. `get_text(strip=True)` gives a tag's text without spaces at its ends. The given cell does this for every row and builds a DataFrame, as the chapter's Strategy 2 does. It keeps empty cells, and prints any row it skips.

```
# wiki_rows: every "tr" in wikitable; print how many
wiki_rows = wikitable.find_all("tr")
print(len(wiki_rows))
```

```
51
```

```
# header_cells: the first row's cells, wiki_rows[0].find_all(["th", "td"]); print how many
header_cells = wiki_rows[0].find_all(["th", "td"])
print(len(header_cells))
```

```
6
```

```
# Print the text of header_cells[3], with get_text(strip=True)
print(header_cells[3].get_text(strip=True))
```

```
Worldwide gross
```

```
# Given: the same for every row. A list comprehension, [... for cell in cells], makes a
# list, one item a cell. A row whose cell count differs from the header's is printed, skipped
header = [cell.get_text(strip=True) for cell in header_cells]
by_hand_rows = []
for row in wiki_rows[1:]:
    values = [cell.get_text(strip=True) for cell in row.find_all(["th", "td"])]
    if len(values) != len(header):
        print("Skipping a row with", len(values), "cells:", values[:3])
        continue
    by_hand_rows.append(values)
by_hand = pd.DataFrame(by_hand_rows, columns=header)
print(len(by_hand), "rows")
print("By hand:   ", list(by_hand.columns))
print("read_html: ", list(films.columns[:6]))
```

```
50 rows
By hand:    ['Rank', 'Peak', 'Title', 'Worldwide gross', 'Year', 'Ref']
read_html:  ['Rank', 'Peak', 'Title', 'Worldwide gross', 'Year', 'Ref']
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Do: print the first three titles from each table, `films["Title"]` and `by_hand["Title"]`. **Share:** what differs, and what it would do if you joined the two tables on `Title`.

```
# Microsprint: print films["Title"].head(3), then by_hand["Title"].head(3)
print(films["Title"].head(3))
print(by_hand["Title"].head(3))
```

```
0            Avengers: Endgame †
1                         Avatar
2    Spider-Man: Brand New Day †
Name: Title, dtype: str
0            Avengers: Endgame†
1                        Avatar
2    Spider-Man: Brand New Day†
Name: Title, dtype: str
```

💬 Discuss

- `get_text(strip=True)` strips the spaces from each piece of text in a cell, then joins the pieces with nothing, so `Endgame †` became `Endgame†`. What would Step 10's `get_text(" ", strip=True)` give?
- Our loop reads every cell as text, `Rank` and `Year` included. What would you do before plotting them?
- No row was skipped here. On what kind of page would the check matter?

Step 10 — An infobox, one row by hand (10 minutes)

An **infobox** is the box of facts at the top right of a Wikipedia article. Open *The Godfather* and Inspect it (week 5): a `<table>` with the class `infobox`, one fact a row, its label in a `<th>` and its value in a `<td>`. Fetch the article, look, find the table, and read one row by hand. `get_text(" ", strip=True)` puts a space between a cell's pieces. The given `read_infobox()` reads every row (chapter, Strategy 4).

```
# godfather_en: request GODFATHER_EN with headers=HEADERS
# Print its status_code and the length of its text
godfather_en = requests.get(GODFATHER_EN, headers=HEADERS)
print(godfather_en.status_code, len(godfather_en.text))
```

```
200 1293568
```

```
# Given, only if the cell above didn't print 200: remove the "#" below (saved 2026-11-11)
# godfather_en = requests.get(saved + "godfather_en.html", headers=HEADERS)
```

```
# en_soup: BeautifulSoup(godfather_en.text, "html.parser")
# infobox_en: its first "table" with class_="infobox"
en_soup = BeautifulSoup(godfather_en.text, "html.parser")
infobox_en = en_soup.find("table", class_="infobox")
```

```
# en_rows: every "tr" in infobox_en; print how many
en_rows = infobox_en.find_all("tr")
print(len(en_rows))
```

```
18
```

```
# One row by hand: print the text of en_rows[2].find("th"),
#   then of en_rows[2].find("td"), with get_text(" ", strip=True)
print(en_rows[2].find("th").get_text(strip=True))
print(en_rows[2].find("td").get_text(" ", strip=True))
```

```
Directed by
Francis Ford Coppola
```

```
# Given: read_infobox(table) does that for every row and returns a dictionary, label: value.
# "if label and value" skips a row without both cells, such as the title or the poster
def read_infobox(table):
    facts = {}
    for row in table.find_all("tr"):
        label = row.find("th")
        value = row.find("td")
        if label and value:
            facts[label.get_text(strip=True)] = value.get_text(" ", strip=True)
    return facts
```

```
# facts_en: read_infobox(infobox_en); print facts_en["Directed by"]
facts_en = read_infobox(infobox_en)
print(facts_en["Directed by"])
```

```
Francis Ford Coppola
```

```
# Print how many facts it found, len(facts_en), and their labels, list(facts_en)
print(len(facts_en), list(facts_en))
```

```
16 ['Directed by', 'Screenplay by', 'Based on', 'Produced by', 'Starring', 'Cinematography',
 ↪  'Edited by', 'Music by', 'Productioncompanies', 'Distributed by', 'Release dates', 'Running
 ↪  time', 'Country', 'Language', 'Budget', 'Box office']
```

💬 Discuss

- One label reads `Productioncompanies`, with no space. Where did the space go? (Look at that label in the Inspector.)
- Print `facts_en["Starring"]`: the whole cast in one string. What would you have to do before counting films per actor, and what could go wrong?
- Print `facts_en["Budget"]`. What would you do before using it as a number, and what would you lose?

Step 11 — The same pattern on French Wikipedia (12 minutes)

The French article, *Le Parrain*, has an infobox too, and the Inspector shows the class `infobox_v3`. But that class is on a `<div>` around the table, not on the `<table>`. The given line uses a **CSS selector**, a short pattern that names tags, with `select()`: `table:has(caption)` means a table with a `<caption>`, a title, inside it. `read_infobox()` doesn't change (chapter, Strategy 5).

```
# godfather_fr: request GODFATHER_FR with headers=HEADERS
# Print its status_code and the length of its text
godfather_fr = requests.get(GODFATHER_FR, headers=HEADERS)
print(godfather_fr.status_code, len(godfather_fr.text))
```

```
200 748888
```

```
# Given, only if the cell above didn't print 200: remove the "#" below (saved 2026-11-11)
# godfather_fr = requests.get(saved + "godfather_fr.html", headers=HEADERS)
```

```
# fr_soup: BeautifulSoup of godfather_fr.text
# Print its first "table" with class_="infobox_v3"
fr_soup = BeautifulSoup(godfather_fr.text, "html.parser")
print(fr_soup.find("table", class_="infobox_v3"))
```

```
None
```

```
# Given: the first table with a <caption> inside it ([0] takes the first); print the caption
infobox_fr = fr_soup.select("table:has(caption)")[0]
print(infobox_fr.find("caption").get_text(strip=True))
```

```
Données clés
```

```
# facts_fr: read_infobox(infobox_fr); print facts_fr["Réalisation"], with its accent
facts_fr = read_infobox(infobox_fr)
print(facts_fr["Réalisation"])
```

```
Francis Ford Coppola
```

```
# Given: save both articles in this folder, as the course did for its saved copies
with open("godfather_en.html", "w", encoding="utf-8") as f:
    f.write(godfather_en.text)
with open("godfather_fr.html", "w", encoding="utf-8") as f:
    f.write(godfather_fr.text)
```

⏱️ Microsprint (5 minutes, alone or in pairs)

Do: the given cell fetches the Spanish article, *El padrino*. Which anchor finds its infobox: the English one, `find("table", class_="infobox")`, or the French one, `select("table:has(caption)")[0]`? Try both. **Share:** the anchor that works, how many facts it gives, and what the other one found.

```
# Given: fetch the Spanish article, look, and save it as the course did
godfather_es = requests.get(GODFATHER_ES, headers=HEADERS)
print(godfather_es.status_code, len(godfather_es.text))
es_soup = BeautifulSoup(godfather_es.text, "html.parser")
with open("godfather_es.html", "w", encoding="utf-8") as f:
    f.write(godfather_es.text)
```

```
200 667503
```

```
# Given, only if the cell above didn't print 200: remove the two "#" below (saved 2026-11-11)
# godfather_es = requests.get(saved + "godfather_es.html", headers=HEADERS)
# es_soup = BeautifulSoup(godfather_es.text, "html.parser")
```

```
# Microsprint: infobox_es, with the English anchor; facts_es = read_infobox(infobox_es)
# Print len(facts_es) and facts_es["Dirección"]
infobox_es = es_soup.find("table", class_="infobox")
facts_es = read_infobox(infobox_es)
print(len(facts_es), facts_es["Dirección"])
```

```
30 Francis Ford Coppola
```

```
# Microsprint: caption_es, the French anchor; print its caption and its number of facts
caption_es = es_soup.select("table:has(caption)")[0]
print(caption_es.find("caption").get_text(strip=True))
print(len(read_infobox(caption_es)))
```

```
Calificaciones profesionales
```

0

💬 **Discuss**

- In week 5 the element picker labeled the CU Boulder article's infobox `table#mwCQ.infobox.vcard`: the class was on the table. On French Wikipedia, what would the picker's label have warned you about?
- `table:has(caption)` found the French infobox because it comes first on the page. What does an anchor that works "because it comes first" promise about next year?
- The English infobox gave 16 facts and the Spanish one 30, about one film. Which of them is the record of the film, if any?

End of Part 2

On Friday the data isn't in a table at all: it's in boxes that repeat down a page. We read one box by hand, let a given function read every box on several pages, and then meet a page whose boxes aren't in its HTML. Part 3 starts with its own setup step, and needs nothing from today.

Part 3 — Friday: boxes into rows, and a page with nothing in it

Many pages show records as **cards**: boxes that repeat the same layout, one for each thing. Each card is a row of a table that the page never drew as a table.

Step 12 — Set up again (3 minutes)

Part 3 needs nothing from Monday or Wednesday. Put your email address in `CLIENT` and run the cell. Open Quotes to Scrape in Chrome beside this notebook. The company Zyte runs it for scraping practice, with quotes from Goodreads, and it has no `robots.txt` (Step 1 got a 404).

```
# Given: Part 3's setup. Put your email address in CLIENT, then run it
import time

import requests
import pandas as pd
import matplotlib.pyplot as plt
from bs4 import BeautifulSoup

CLIENT = "WebDataScience/1.0 (your-email@colorado.edu)"
HEADERS = {"User-Agent": f"{CLIENT} python-requests/{requests.__version__}"}
QUOTES_URL = "https://quotes.toscrape.com/"
saved =
 ↪  "https://raw.githubusercontent.com/cuinfoscience/INFO4617-Fall2026/main/labs/week-06/data/"
```

💬 **Discuss**

- Quotes to Scrape exists to be scraped. What can you practise on it that a real site wouldn't let you, and what can't it teach you?

Step 13 — One card by hand (10 minutes)

Right-click a quote in Chrome and choose Inspect (week 5). Each card is a `<div class="quote">`, with the quote in a `<span class="text">`, the author in a `<small class="author">`, and each of its tags, the topic words under it, in an `<a class="tag">`. Fetch the page, look, find the cards, and read the first one by hand (chapter, Strategy 6).

```
# quotes_page: request QUOTES_URL with headers=HEADERS
# Print its status_code and the length of its text
```

```
quotes_page = requests.get(QUOTES_URL, headers=HEADERS)
print(quotes_page.status_code, len(quotes_page.text))
```

```
200 11021
```

```
# quotes_soup: BeautifulSoup of quotes_page.text
# cards: every "div" with class_="quote" in it; print how many
quotes_soup = BeautifulSoup(quotes_page.text, "html.parser")
cards = quotes_soup.find_all("div", class_="quote")
print(len(cards))
```

```
10
```

```
# first_card: cards[0]. Print the text of its "span" with class_="text"
first_card = cards[0]
print(first_card.find("span", class_="text").get_text(strip=True))
```

```
“The world as we have created it is a process of our thinking. It cannot be changed without
↪  changing our thinking.”
```

```
# Print the text of first_card's "small" with class_="author"
print(first_card.find("small", class_="author").get_text(strip=True))
```

```
Albert Einstein
```

```
# first_tags: every "a" with class_="tag" in first_card
# Print how many, and the text of the first one
first_tags = first_card.find_all("a", class_="tag")
print(len(first_tags), first_tags[0].get_text(strip=True))
```

```
4 change
```

💬 Discuss

- One card is one row. What columns would you give the table, and which of them holds more than one value?
- Week 4's XML named its contents in its tags, such as `<party>`. How good a guide are this page's class names, by comparison?

Step 14 — Every card, every page (12 minutes)

The given function `read_quotes_page(number)` does what you just did, for every card on one page of the site (`/page/2/` and so on), and returns a DataFrame, one row a card. Find your lines in it, then call it for page 2. The given loop reads pages 1 to 3 a second apart, with `time.sleep(1)`, and stacks them with `pd.concat()`. Then look for the card with no tags.

```
# Given: read every card on one page into a DataFrame, one row a card.
# ", ".join(tags) puts the tags in one cell, separated by commas; n_tags counts them
def read_quotes_page(number):
    page = requests.get(f"{QUOTES_URL}page/{number}/", headers=HEADERS)
    print("page", number, ":", page.status_code, len(page.text))
    soup = BeautifulSoup(page.text, "html.parser")
    rows = []
    for card in soup.find_all("div", class_="quote"):
        tags = []
        for tag in card.find_all("a", class_="tag"):
            tags.append(tag.get_text(strip=True))
        rows.append({"text": card.find("span", class_="text").get_text(strip=True),
                     "author": card.find("small", class_="author").get_text(strip=True),
```

```
                    "tags": ", ".join(tags), "n_tags": len(tags), "page": number})
    return pd.DataFrame(rows)
```

```
# page_two: read_quotes_page(2); print how many rows it has
page_two = read_quotes_page(2)
print(len(page_two))
```

```
page 2 : 200 13699
10
```

```
# Given: pages 1 to 3, a second apart, stacked into one table; save it here as quotes.csv
quote_pages = []
for number in [1, 2, 3]:
    quote_pages.append(read_quotes_page(number))
    time.sleep(1)
quotes = pd.concat(quote_pages, ignore_index=True)
quotes.to_csv("quotes.csv", index=False)
print(len(quotes), "quotes from", quotes["author"].nunique(), "authors")
```

```
page 1 : 200 11021

page 2 : 200 13699

page 3 : 200 9987

30 quotes from 20 authors
```

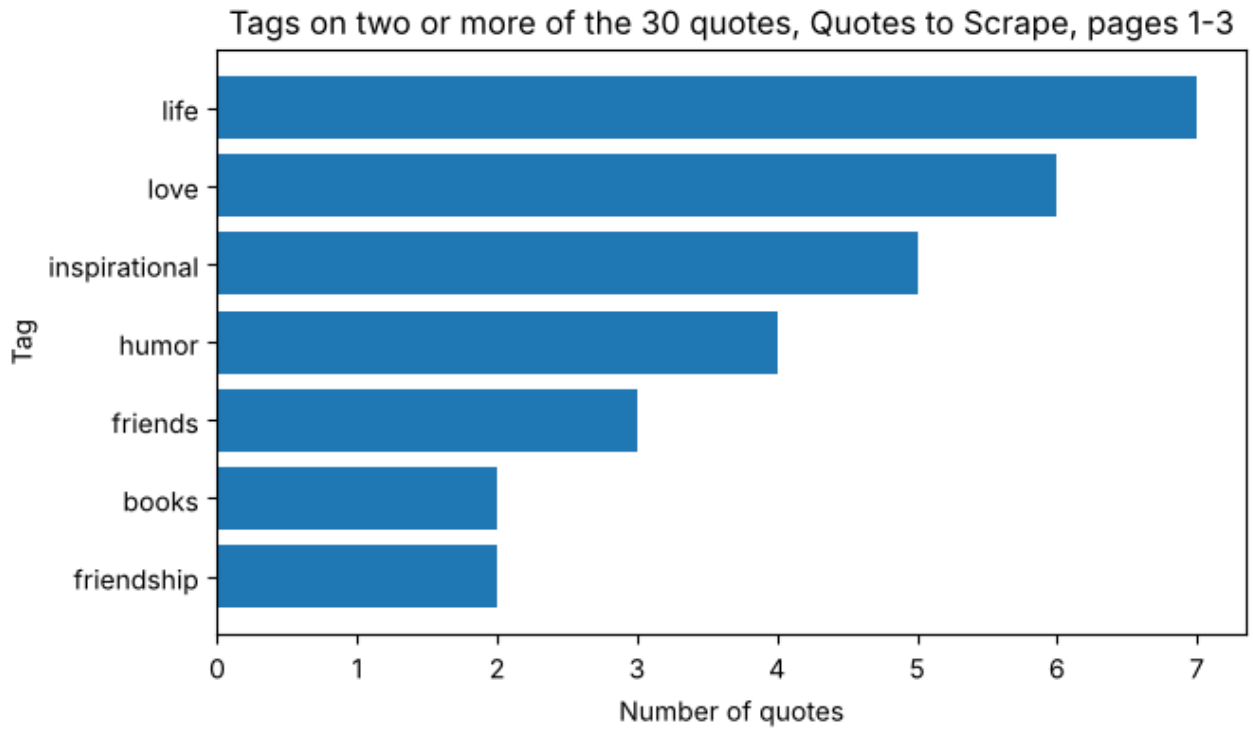
```
# Given, only if the cell above stopped: remove the "#" below (saved 2026-11-11)
# quotes = pd.read_csv(saved + "quotes.csv", keep_default_na=False)
```

```
# Show the quotes with no tags: quotes[quotes["n_tags"] == 0]
quotes[quotes["n_tags"] == 0]
```

```
                                                 text        author tags  \
27  “It is impossible to live without failing at s...  J.K. Rowling

    n_tags  page
27       0     3
```

```
# Given: count each tag across the quotes; draw those on two or more quotes as horizontal bars.
# split(", ") makes "love, life" a list; explode() gives each item a row; "" is no tag
all_tags = quotes["tags"].str.split(", ").explode()
tag_counts = all_tags[all_tags != ""].value_counts()
tag_counts = tag_counts[tag_counts >= 2].sort_values()
fig, ax = plt.subplots(figsize=(7, 4))
ax.barh(tag_counts.index, tag_counts.values)
ax.set_title("Tags on two or more of the 30 quotes, Quotes to Scrape, pages 1-3")
ax.set_xlabel("Number of quotes")
ax.set_ylabel("Tag")
plt.show()
```



⏱️ **Microsprint (4 minutes, alone or in pairs)**

Do: the site's last page is 10. Call `read_quotes_page(11)`, and open page 11 in Chrome. **Share:** its status, its number of rows, what Chrome shows, and how a loop over pages could know when to stop.

```
# Microsprint: page_eleven: read_quotes_page(11); print how many rows it has
page_eleven = read_quotes_page(11)
print(len(page_eleven))
```

```
page 11 : 200 3048
0
```

💬 **Discuss**

- Page 11 answered 200, with no quotes and no error. For a program, is that better or worse than a 404 would have been?
- Our function assumes every request works and every card has a text and an author. What would it do with a 429 for page 2, or a card with no author, and how do the chapter's `responsible_get()` and `safe_scrape()` guard against each?
- The card with no tags became a row with an empty `tags` cell. Is empty the same as missing?

Step 15 — When the data isn't in the page (5 minutes)

Every page so far sent its data in its HTML. Request the same site's `/js/` page, which looks the same in Chrome. Look at what came back, count the cards, and ask whether one author's name appears anywhere in the text. In Chrome, compare View Source with the Inspector (week 5). This is the chapter's Strategy 8: a page that isn't static, whose content arrives only after a script runs.

```
# js_page: request QUOTES_URL + "js/" with headers=HEADERS
# Print its status_code and the length of its text
js_page = requests.get(QUOTES_URL + "js/", headers=HEADERS)
print(js_page.status_code, len(js_page.text))
```

```
200 5806
```

```
# js_soup: BeautifulSoup of js_page.text; print how many "div"s with class_="quote" it has
js_soup = BeautifulSoup(js_page.text, "html.parser")
print(len(js_soup.find_all("div", class_="quote")))
```

```
0
```

```
# Is "Albert Einstein" anywhere in js_page.text? Print the answer, True or False
print("Albert Einstein" in js_page.text)
```

```
True
```

💬 **Discuss**

- The page came back with 200 and no cards, yet Einstein's name is in the text. Where is it (View Source), and what turns it into cards in your browser?
- The chapter's Rotten Tomatoes grid is built from tags the site made up, `<media-info-tile>` and `<rt-text slot="criticsScore">`. Could `read_quotes_page()` read it, and what would you need instead?
- Place this week's pages on the chapter's 2×2 of single or multiple sites and consistent or inconsistent design. Which quadrant needs the most code?
- Typed in lower case, `"albert einstein" in js_page.text` is `False`, with no error. What exactly does `in` check?

Step 16 — What did we learn? (5 minutes)

Answer in a few sentences, as comments in the cell below:

1. Which sites' `robots.txt` let us fetch the chapter's page, and why did this lab still send three of them nothing but that one request?
2. What did the chapter's cleanup do to one film's gross, and what check caught it? Which column has the same problem with no `$` to anchor on?
3. Which anchor found the infobox on each of the three Wikipedias?
4. How many quotes did the given loop collect, and what did page 11 and `/js/` give back?

Recommended Exercises ahead (in the chapter's companion notebook, due Sunday). Choose a table other than the lab's list of highest-grossing films.

- Step 1, choose and check: Step 1's `robots.txt` by hand and its given loop; Step 3's request with `HEADERS`.
- Step 2, cache the raw HTML: Step 3's `with open(...)`, to write the file and to read it back.
- Step 3, extract with `read_html()`: Step 4's `read_html(StringIO(...))`, the count, and the index.
- Step 4, clean it: Step 5's `dtypes` and the given cleanups of Steps 5–6, with the checks that tested them.
- Step 5, visualize: the titled, labeled plots of Steps 5 and 14; Step 14's is a horizontal bar chart (yours ranks the rows of your table).
- Step 6, look under the hood: Step 8's `find("table", class_="wikitable")` and Step 9's header row, read with `find_all(["th", "td"])`.
- Step 7, interpret: Steps 8, 9 and 11, where a class matched nothing, two tables disagreed by a space, and a class sat on another tag.

```
# Your answers here, as comments
# 1. Wikipedia's said yes to the list, and so did oscars.org's, Rotten Tomatoes' and Quotes to
 ↪  Scrape's
#    (a 404: no file, so no rules); Box Office Mojo's and IMDb's said False. The Numbers refused
 ↪  the
#    robots.txt request itself (403), so its True means nothing. Their terms, not robots.txt, are
 ↪  why
#    The Numbers, oscars.org and Rotten Tomatoes got nothing else from us: all three forbid
 ↪  scraping.
```

```
# 2. It turned The Fate of the Furious's F8$1,238,764,765 into 81,238,764,765, about 81 billion
#    dollars: the footnote's 8 became a digit. The plot showed one point far above the rest, and
 ↪  the
#    sort found it. Keeping what follows the "$" changed that row only. Peak has 24RK, 23RK, 5RK
 ↪  and
#    4TS3, which the digits-only cleanup would make 24, 23, 5 and 43, and Peak has no "$".
# 3. English: find("table", class_="infobox"), 16 facts. French: infobox_v3 was on a div, so
#    select("table:has(caption)")[0]. Spanish: the English anchor, 30 facts; the French one found
 ↪  a
#    ratings table, "Calificaciones profesionales", with 0 facts. All three gave Francis Ford
 ↪  Coppola.
# 4. 30 quotes from 20 authors on pages 1 to 3; J.K. Rowling's card on page 3 had no tags. Page
 ↪  11
#    answered 200 with no quotes and no error. /js/ answered 200 with no quote cards, but "Albert
#    Einstein" is in its text: the quotes are inside a script, which a browser runs and requests
 ↪  doesn't.
```

Friday: from the lab to the textbook (15 minutes)

What did the lab turn up that the chapter should know? Some places to start:

- **Steps 1–2:** the chapter scrapes The Numbers, oscars.org and Rotten Tomatoes, whose terms forbid it, and says oscars.org "lets you in" if you identify yourself. What should it use, or say, instead?
- **Step 4:** what `read_html()` says about a text with no table when `html5lib` isn't installed. The chapter doesn't mention it.
- **Steps 5–6:** the chapter's digits-only cleanup, The Fate of the Furious, and `Peak`'s `4TS3`. Is a fix, a warning, or a check the better revision?
- **Steps 9 and 11:** `get_text(strip=True)` against `read_html()`'s titles; or `table:has(caption)`, which matches two tables on the French page.
- **A microsprint** that would make a good exercise, such as page 11.

`handouts/common/revision-framework.md` says how to propose a revision.